

Algorithms in Computational Geometry – Convex Hull

For this honors project, you will be implementing the **divide-and-conquer** algorithm for computing a convex hull from a set of points.

1 Introduction

Accompanying this assignment file on HuskyCT is a zip file `convexhull.zip`, which contains a `tkinter` GUI called `hullGUI.py`. This GUI allows you to click in a window to add points, and then click a button to compute and draw the convex hull around those points. Note, in this assignment the points are actually pictures of the original UConn Husky logo which is both funny and terrifying. You are free to change this image; in any case, the point is taken as the center pixel. It also contains `divide_and_conquer_hull.pdf` which contains a description of the algorithm.

You will submit the file `convexhull.py` which is also in the archive. It contains several functions that you will find useful (including trickier geometric tests, e.g. a function for sorting a set of points counter-clockwise – you should probably not modify these), as well as a function, `computeHull(points)`, which takes a list of points and returns a list of points in the convex hull (in clockwise order). The implementation of `computeHull(points)` in the archive is incorrect because it simply returns all the points, not just those in the convex hull.

The representation of a point is a 2-tuple of integers, i.e. (x,y) . As was the case with homework 1, the upper-left of the drawing canvas is the origin $(0,0)$. One implication of this is that all of the coordinates are positive.

Your task is to **correctly implement** `computeHull(points)` **and then benchmark its running time**.

You may of course write any supplementary functions you deem useful but they must be implemented inside `convexhull.py`. Any additional resource that you use must also be included in your submission so that we can run your implementation. Also, write a separate function for your *base case*, which should be the naive algorithm. Call it whenever you need to compute the hull on fewer than a constant number of points (e.g. 4 points). You will have to benchmark this naive implementation on large inputs as well for benchmarking.

2 Implementation

Your solution should be written in Python. We will evaluate your solutions in Python 2.7. Note that if you use Python 3.x, you should change (in `hullGUI.py`):

```
from Tkinter import *  
  
to  
  
from tkinter import *
```

3 Due date

Because we have to be able to evaluate your work before final grades are due, submissions will not be accepted after the due date of Final Exam Day.

4 Grading Rubric

Your grade will be based on three components:

- Functional Correctness (50%)
- Design and Representation (15%)
- Invariant Documentation (10%)
- Benchmarking and Analysis (25%)

Functional Correctness means passing test cases. Remember: your function `computeHull(points)` must return the points in the convex hull *in clockwise order*. Although we do not provide test cases, you can use the GUI for small tests or write up unit tests yourself. For this you will need a way to programmatically generate points. Given that you must also benchmark your implementation, it is probably a good idea to do this regardless of tests for correctness.

Note: *you may modify `hullGUI.py` in any way to support debugging or testing, but you need not submit it through HuskyCT.*

Design and Representation is our evaluation of the structure of your program, the choice of representations (de-emphasized here because some choices have already been made for you), and the use of appropriate (not excessive) comments.

Invariant Documentation is to force you to reason about the running time of the algorithm, as well as its correctness.

- Initialization: how is the problem set up

- Maintenance: how do I know I'm making progress?
- Termination: how do I know I'm done
- Usually, these should all be closely related.
- Properly documenting invariants can save you a great deal of time thinking about your algorithm.

Benchmarking and Analysis is similar to assignment 1, but you need to come up with reasonable inputs. You should include a PDF in your upload to HuskyCT, along with your source code.

- Implement a way to generate a certain number of points (for example, if you decide to benchmark on a million points, you need to generate those points at random). You should **not** use the GUI for this. The GUI is there for you to play with, but proper benchmarking should be non-interactive.
- You must *also* benchmark your base-case (which should actually work for any reasonably small number of points) and plot its running time.
- Benchmark your implementation on a wide enough set of inputs that you can plot a meaningful curve.
- Write a brief summary of your benchmarking results. Do they support your expectation of the asymptotic complexity of this algorithm? Why or why not?